

# Passenger Assignment Model in a Public Transportation Network

Michel Bierlaire

Evangelos Paschalidis

July 28, 2026

## 1 Introduction

This document describes the modeling assumptions, mathematical formulation, and inference components implemented in the software. It is intended as technical documentation for users and developers of the package.

The objective of this software is to infer an aggregate representation of public transportation demand from operational data. The demand is characterized by a flow for each origin–destination (OD) pair and each desired departure time interval.

The software follows four main steps:

1. Develop a detailed network representation of the public transportation system.
2. Formulate an assignment model that maps an OD matrix to passenger flows on the network.
3. Define measurement equations linking model outputs to observed data, yielding a likelihood function.
4. Estimate demand using ML, MAP, or Bayesian variational inference.

## 2 Modeling Elements

The following sections document the mathematical objects and modeling assumptions used internally by the software.

### 2.1 Spatial Representation

Let  $\mathcal{S}$  denote the set of stops in the area of interest. Each stop can act as an origin, a destination, or a transfer location.

**Definition 1** (OD pair). *An origin–destination pair is a tuple*

$$\mathbf{q} = (s_o, s_d) \in \mathcal{Q} = \mathcal{S} \times \mathcal{S}.$$

### 2.2 Temporal Representation

The analysis period is partitioned into a set  $\mathcal{T}$  of departure time intervals. Each interval represents a desired departure time window.

**Definition 2** (Departure time interval). *Each time bin  $\mathbf{t} \in \mathcal{T}$  corresponds to a closed interval*

$$[\mathbf{a}_{\mathbf{t}}, \mathbf{b}_{\mathbf{t}}] \subset \mathbb{R}, \quad \mathbf{a}_{\mathbf{t}} < \mathbf{b}_{\mathbf{t}},$$

*where  $\mathbf{a}_{\mathbf{t}}$  and  $\mathbf{b}_{\mathbf{t}}$  denote the start and end time (in minutes) of the desired departure interval.*

For each departure interval  $\mathbf{t} \in \mathcal{T}$ , the desired departure window is the interval  $[\mathbf{a}_{\mathbf{t}}, \mathbf{b}_{\mathbf{t}}]$ .

**Definition 3** (OD demand). *For each OD pair  $\mathbf{q} \in \mathcal{Q}$  and departure time interval  $\mathbf{t} \in \mathcal{T}$ , let*

$$\mathbf{f}_{\mathbf{q}}^{\mathbf{t}} \geq 0$$

*denote the number of passengers wishing to depart during interval  $\mathbf{t}$  from origin  $\mathbf{s}_{\mathbf{o}}$  to destination  $\mathbf{s}_{\mathbf{d}}$ .*

The full demand vector is denoted by

$$\mathbf{f} = \{\mathbf{f}_{\mathbf{q}}^{\mathbf{t}} : \mathbf{q} \in \mathcal{Q}, \mathbf{t} \in \mathcal{T}\}.$$

## 2.3 Baseline demand and deviation parameterization (Bayesian layer)

The model assumes that the analyst can provide an *a-priori* demand matrix  $\mathbf{f}^{(0)} = \{\mathbf{f}_{\mathbf{q}}^{\mathbf{t},(0)}\}$ , aligned with the OD/time-bin indexing of the scenario. Rather than placing a prior directly on the demand levels  $\mathbf{f}$ , we parameterize demand in terms of *log-deviations* from  $\mathbf{f}^{(0)}$ . This improves numerical stability, facilitates weakly informative priors, and makes the role of prior information explicit.

**Modeling choice (smoothly bounded log-deviation).** For each OD/time-bin cell, introduce an unconstrained latent variable  $\tilde{\mathbf{z}}_{\mathbf{q}}^{\mathbf{t}} \in \mathbb{R}$ . To protect the exponentiation and the downstream assignment from extreme values while retaining a smooth gradient, define the effective log-deviation as

$$\mathbf{z}_{\mathbf{q}}^{\mathbf{t}} = \mathbf{z}_{\max} \tanh\left(\frac{\tilde{\mathbf{z}}_{\mathbf{q}}^{\mathbf{t}}}{\mathbf{z}_{\max}}\right), \quad \mathbf{z}_{\max} > 0.$$

The demand used by the assignment is then

$$\mathbf{f}_{\mathbf{q}}^{\mathbf{t}} = \mathbf{f}_{\mathbf{q}}^{\mathbf{t},(0)} \exp(\mathbf{z}_{\mathbf{q}}^{\mathbf{t}}).$$

This transformation guarantees  $\mathbf{f}_{\mathbf{q}}^{\mathbf{t}} \geq 0$  whenever  $\mathbf{f}_{\mathbf{q}}^{\mathbf{t},(0)} \geq 0$  and preserves the baseline structure up to multiplicative adjustments. It also implies

$$|\mathbf{z}_{\mathbf{q}}^{\mathbf{t}}| \leq \mathbf{z}_{\max}, \quad \exp(-\mathbf{z}_{\max}) \leq \frac{\mathbf{f}_{\mathbf{q}}^{\mathbf{t}}}{\mathbf{f}_{\mathbf{q}}^{\mathbf{t},(0)}} \leq \exp(\mathbf{z}_{\max})$$

for positive baseline cells. The default  $\mathbf{z}_{\max} = 6$  therefore permits demand multipliers ranging approximately from 1/403 to 403. In the software configuration, this smooth bound is exposed under the historical name `z_clip`; despite that name, the implementation uses a smooth hyperbolic-tangent transformation rather than hard clipping.

**Zero baseline cells.** If  $\mathbf{f}_{\mathbf{q}}^{\mathbf{t},(0)} = 0$ , then  $\mathbf{f}_{\mathbf{q}}^{\mathbf{t}} = 0$  for all values of  $\tilde{\mathbf{z}}_{\mathbf{q}}^{\mathbf{t}}$ . Therefore, OD cells that are zero in the baseline matrix remain structurally zero during inference. In order to allow strictly positive deviations from zero, a small but non zero seed value must be provided:  $\mathbf{f}_{\mathbf{q}}^{\mathbf{t},(0)} = \varepsilon > 0$ .

**Prior scale.** The raw unconstrained variables, rather than the bounded effective deviations, are assumed independent and normally distributed:

$$\tilde{\mathbf{z}}_q^t \sim \mathcal{N}(0, \sigma_z^2).$$

The parameter  $\sigma_z > 0$  controls the strength of regularization. Small values of  $\sigma_z$  constrain demand to remain close to the baseline, while large values allow the effective deviations to approach their smooth bounds. In the current version of the software,  $\sigma_z$  is a fixed hyperparameter.

The distinction between  $\tilde{\mathbf{z}}_q^t$  and  $\mathbf{z}_q^t$  is important: the Gaussian density is evaluated at the raw variable  $\tilde{\mathbf{z}}_q^t$ , whereas the assignment and all reported demand samples use the transformed value  $\mathbf{z}_q^t$ .

**Separation of responsibilities.** The assignment procedure is treated as a deterministic mapping

$$(\mathbf{f}, \theta) \mapsto \mathbf{x}(\mathbf{f}; \theta)$$

and remains agnostic to priors and baseline information. The construction of  $\mathbf{f}$  from  $\mathbf{f}^{(0)}$  and  $\tilde{\mathbf{z}}$  is performed in the *Bayesian layer* (implemented using NumPyro and stochastic variational inference), which: (i) reads the baseline  $\mathbf{f}^{(0)}$  provided by the user, (ii) draws the raw deviations  $\tilde{\mathbf{z}}$  from their prior distribution, (iii) applies the smooth bound to obtain  $\mathbf{z}$ , and (iv) computes  $\mathbf{f}$  using the expression above before invoking the assignment. This design avoids introducing additional indexing conventions and keeps the assignment module independent of the inference engine.

**Desired departure interval and admissible boarding window.** For each OD pair  $q \in \mathcal{Q}$  and departure time interval  $t \in \mathcal{T}$ , we associate the desired departure *interval*  $[\mathbf{a}_t, \mathbf{b}_t]$  and an admissible schedule-deviation window of half-width  $\Delta_{\max}^{\text{access}} > 0$  (user-defined, e.g. 15 minutes).

**Admissible access links.** An access link is included in the graph *only if* the departure time  $\tau$  is not too far from the desired interval, namely

$$\tau \in [\mathbf{a}_t - \Delta_{\max}^{\text{access}}, \mathbf{b}_t + \Delta_{\max}^{\text{access}}],$$

where  $\tau$  denote the actual departure time (in minutes) of a boarding at the origin stop. As described later, the boarding is modeled by a link, that is included if the distance from  $\tau$  to the interval  $[\mathbf{a}_t, \mathbf{b}_t]$  does not exceed  $\Delta_{\max}^{\text{access}}$ .

**Schedule-deviation penalty.** When the access link is included, its generalized cost is defined by a (possibly asymmetric) piecewise-linear penalty with a *flat* (zero-penalty) region inside the interval:

$$c_{\text{access}}(\tau; t) = \beta_{\text{early}} \max(0, \mathbf{a}_t - \tau) + \beta_{\text{late}} \max(0, \tau - \mathbf{b}_t),$$

where  $\beta_{\text{early}} \geq 0$  and  $\beta_{\text{late}} \geq 0$  are user-defined coefficients. Therefore,

$$c_{\text{access}}(\tau; t) = 0 \quad \text{whenever} \quad \tau \in [\mathbf{a}_t, \mathbf{b}_t],$$

and penalties apply only to departures strictly earlier than  $\mathbf{a}_t$  or strictly later than  $\mathbf{b}_t$ .

Note that the thresholds  $\Delta_{\max}^{\text{access}}$  and  $\Delta_{\max}^{\text{transfer}}$  are fixed configuration parameters; the assignment is differentiable w.r.t. costs and OD flows given a fixed graph.

**Implementation note (departure interval as OD/time-bin data).** Although centroid-in nodes are not time-tagged in the graph, the desired departure *interval*  $[\mathbf{a}_t, \mathbf{b}_t]$  remains an attribute of the OD/time-bin pair. In computations, the access-link penalty is evaluated by combining the interval bounds  $(\mathbf{a}_t, \mathbf{b}_t)$  with the clock time  $\tau$  of the *departure event node* reached by that access link. This yields a zero penalty for departures within  $[\mathbf{a}_t, \mathbf{b}_t]$  and an early/late penalty based on the nearest bound otherwise, without introducing backward-in-time links in the acyclic time-expanded network.

## 2.4 Network Representation

The public transportation system is represented as a directed time-expanded graph. All timetable times (arrival/departure event times) are expressed in minutes on the same time axis as the departure-time intervals.

A key modeling choice is (i) the separation of each stop centroid into two distinct nodes (an origin *centroid-in* and a destination *centroid-out*), and (ii) the split of each timetable event into an *arrival* and a *departure* node. This second split prevents ill-defined instantaneous moves (e.g., transferring or boarding at the exact same time as arrival) and ensures that all links respect a strict chronological (or lexicographic) order, so that the resulting time-expanded network is acyclic.

**Definition 4** (Nodes). *The node set consists of three families of nodes:*

- **Centroid-in nodes (non time-tagged, indexed by departure interval):** for each stop  $s \in \mathcal{S}$  and for each departure interval  $\mathbf{t} \in \mathcal{T}$ , a node

$$(s, \mathbf{t})^{\text{in}}$$

*used as a demand-injection handle for passengers departing from stop  $s$  with desired departure interval  $\mathbf{t}$ . These nodes do not correspond to a physical time instant; they are placed before all event nodes in the topological order (conceptually at  $-\infty$ ). The interval index  $\mathbf{t}$  is used only to (i) select admissible boarding events through the access-window rule, and (ii) compute schedule-deviation penalties.*

- **Centroid-out nodes (non time-tagged):** for each stop  $s \in \mathcal{S}$ , a node  $s^{\text{out}}$  representing the point where passengers arriving at stop  $s$  exit the network. These nodes are not associated with a physical time; they are placed after all event nodes in the topological order (conceptually at  $+\infty$ ) so that the global graph remains acyclic.
- **Timetable event nodes (trip-specific, split into arrival and departure):** for each stop  $s$ , each scheduled arrival time  $\tau^{\text{arr}}$  and departure time  $\tau^{\text{dep}}$  associated with a given vehicle run (trip), we create two nodes

$$(s, \tau^{\text{arr}})^{\text{arr}} \quad \text{and} \quad (s, \tau^{\text{dep}})^{\text{dep}}.$$

*These nodes are trip-specific: if two trips serve the same stop at the same clock time, they still correspond to different event nodes.*

**Definition 5** (Lines and event-to-line mapping). *Let  $\mathcal{L}$  denote the set of public transport lines. Each timetable event node (arrival or departure) is associated with exactly one operated line, captured by the mapping*

$$\lambda : \mathcal{V}^{\text{event}} \rightarrow \mathcal{L}, \quad v \mapsto \lambda(v),$$

*where  $\mathcal{V}^{\text{event}}$  is the set of timetable event nodes. In practice,  $\lambda(v)$  is derived from the timetable: event nodes created from a given trip inherit the line identifier of that trip. Therefore, for a given trip at stop  $s$ , the corresponding arrival and departure nodes share the same line label.*

Centroid-in nodes are not time-tagged: they are placed before all timetable event nodes (conceptually at  $-\infty$ ) so that access links can connect to departure events that occur either before or after the desired departure window without violating acyclicity. Centroid-out nodes are not time-tagged and are placed after all event nodes (conceptually at  $+\infty$ ), preserving a global topological order.

**Remark.** The separation of centroid-in and centroid-out nodes is essential for the validity of the forward dynamic-programming loading algorithm. Without this separation, egress links could break the acyclic structure and violate the topological order implied by time.

**Definition 6 (Links).** *The link set  $\mathcal{A}$  contains the following link types:*

1. **Access links (time-windowed):** for an OD pair  $\mathbf{q} = (s_o, s_d)$  and departure interval  $\mathbf{t}$ , access links connect the origin centroid-in node  $(s_o, \mathbf{t})^{\text{in}}$  to selected departure event nodes at the origin stop,

$$(s_o, \mathbf{t})^{\text{in}} \rightarrow (s_o, \tau)^{\text{dep}},$$

only for departure times  $\tau$  satisfying

$$\tau \in [a_t - \Delta_{\text{max}}^{\text{access}}, b_t + \Delta_{\text{max}}^{\text{access}}].$$

2. **Ride links:** for each trip, ride links connect a departure event at the current stop to an arrival event at the next stop,

$$(s_i, \tau_i^{\text{dep}})^{\text{dep}} \rightarrow (s_{i+1}, \tau_{i+1}^{\text{arr}})^{\text{arr}}.$$

3. **Dwell/continue links (within-trip at a stop):** for each trip and each stop event, a dwell link connects the arrival and departure nodes at the same stop,

$$(s, \tau^{\text{arr}})^{\text{arr}} \rightarrow (s, \tau^{\text{dep}})^{\text{dep}}.$$

In the implementation, strictly positive dwell time is enforced by regularizing timetable records where arrival equals departure (using a small minimum dwell time).

4. **Transfer links (inter-line, time-windowed):** transfer links connect an arrival event node to a later departure event node at the same stop and represent waiting and switching between different lines. A transfer link

$$(s, \tau_1)^{\text{arr}} \rightarrow (s, \tau_2)^{\text{dep}}, \quad \text{with} \quad \tau_2 > \tau_1$$

is included only if

- (a) the two event nodes belong to different lines, and
- (b) the waiting time satisfies  $\tau_2 - \tau_1 \leq \Delta_{\text{max}}^{\text{transfer}}$ .

No transfer links are created between events of the same line. In particular, transfer links are never created for  $\tau_2 = \tau_1$ .

5. **Egress links (global graph, destination-gated by masking):** the global graph contains egress links from every arrival event node at stop  $s$  to the centroid-out node  $s^{\text{out}}$ :

$$(s, \tau)^{\text{arr}} \rightarrow s^{\text{out}}.$$

For a given OD pair  $\mathbf{q} = (s_o, s_d)$ , only the destination centroid-out node  $s_d^{\text{out}}$  is enabled as a terminal node; all other egress options are disabled by a destination-dependent mask.

Each link  $\ell \in \mathcal{A}$  is associated with:

- a generalized cost  $c_\ell$ ,
- a capacity  $u_\ell$  for ride links (stored in the data structure, not yet used in costs).

**Assumption 1** (Acyclic time-expanded network). *All links in the time-expanded network are consistent with a strict topological order. All event-to-event links strictly increase clock time. In addition, centroid-in nodes  $(s, t)^{\text{in}}$  are placed before all event nodes (conceptually at  $-\infty$ ), and centroid-out nodes  $s^{\text{out}}$  are placed after all event nodes (conceptually at  $+\infty$ ). With this convention, access and egress links always respect the global ordering.*

*With the event split, feasible movements follow the pattern*

$$(s, t)^{\text{in}} \rightarrow (s, \tau)^{\text{dep}} \rightarrow (s', \tau')^{\text{arr}} \rightarrow (s', \tau'')^{\text{dep}} \rightarrow \dots \rightarrow (s_d, \bar{\tau})^{\text{arr}} \rightarrow s_d^{\text{out}}.$$

*Transfer links satisfy  $\tau_2 > \tau_1$  and dwell links are enforced to be strictly forward in time through a minimum dwell regularization. Therefore the directed graph is acyclic and admits a topological ordering, which allows value functions and flows to be computed using dynamic programming.*

## 2.5 Generalized Cost

Each passenger seeks to minimize a generalized cost that includes:

- in-vehicle travel time (ride links),
- waiting and transfer time (transfer links) and dwell time at stops (dwell/continue links),
- transfer disutility through coefficient  $\beta_{\text{transfer}}$ ,
- early/late departure penalty relative to the desired departure *interval* (access links; zero penalty within  $[a_t, b_t]$ ).

For a path  $p$ , the generalized cost is

$$C(p) = \sum_{\ell \in p} c_\ell.$$

### 2.5.1 Link-type-specific cost formulation

The generalized cost of each link depends on its type. All coefficients appearing below are fixed user-defined parameters.

**Ride links.** For a ride link  $\ell$  representing the movement of a vehicle between two stops, connecting a departure event to a subsequent arrival event,

$$(s_i, \tau_i^{\text{dep}})^{\text{dep}} \rightarrow (s_{i+1}, \tau_{i+1}^{\text{arr}})^{\text{arr}},$$

the generalized cost is equal to the in-vehicle travel time:

$$c_\ell = \tau_{i+1}^{\text{arr}} - \tau_i^{\text{dep}}.$$

**Dwell/continue links.** For a dwell/continue link at a stop,

$$(s, \tau^{\text{arr}})^{\text{arr}} \rightarrow (s, \tau^{\text{dep}})^{\text{dep}},$$

the generalized cost is the dwell time at the stop:

$$c_\ell = \tau^{\text{dep}} - \tau^{\text{arr}}.$$

**Transfer links (inter-line, bounded waiting).** Consider a transfer link  $\ell : (s, \tau_1)^{\text{arr}} \rightarrow (s, \tau_2)^{\text{dep}}$  with  $\tau_2 > \tau_1$ . Such a link is included only if (i) the two events correspond to *different lines* and (ii) the waiting time  $\tau_2 - \tau_1$  does not exceed the user-defined threshold  $\Delta_{\text{max}}^{\text{transfer}}$ . When included, its generalized cost is

$$c_\ell = \beta_{\text{transfer}} (\tau_2 - \tau_1),$$

where  $\beta_{\text{transfer}} > 0$  is a user-defined coefficient.

**Access links (bounded schedule deviation with a flat region).** Consider an access link from the centroid-in node  $(s_o, t)^{\text{in}}$  to an origin *departure* event node  $(s_o, \tau)^{\text{dep}}$ . The link is included only if

$$\tau \in [a_t - \Delta_{\text{max}}^{\text{access}}, b_t + \Delta_{\text{max}}^{\text{access}}].$$

When included, its generalized cost is

$$c_\ell = \beta_{\text{early}} \max(0, a_t - \tau) + \beta_{\text{late}} \max(0, \tau - b_t),$$

with user-defined coefficients  $\beta_{\text{early}} \geq 0$  and  $\beta_{\text{late}} \geq 0$ .

**Egress links to centroids.** For an egress link  $\ell : (s, \tau)^{\text{arr}} \rightarrow s^{\text{out}}$ , we set  $c_\ell = 0$ .

**Interpretation of cost coefficients.** All time quantities in the model are measured in minutes. Travelers may perceive these minutes differently depending on context. The coefficients  $\beta_{\text{transfer}}$ ,  $\beta_{\text{early}}$  and  $\beta_{\text{late}}$  act as perception weights converting physical time into generalized cost. These coefficients are fixed and specified by the user; they are not estimated within the inference procedure. Only the OD demand and optionally the dispersion parameter  $\theta$  are estimated.

## 3 Assignment Model

### 3.1 Input

The input is the OD vector  $\mathbf{f}$  (scenario order), which is injected through centroid-in nodes.

### 3.2 Assignment Procedure

The classical assignment procedure based on deterministic shortest paths is not differentiable with respect to link costs and is therefore not well suited for gradient-based calibration methods. We adopt a differentiable assignment model inspired by the logit-based approach of Dial (1971), adapted to the acyclic time-expanded public transportation network.

### 3.3 Differentiable Dial-style assignment

Let  $\mathbf{f}$  denote the OD demand. For each OD pair  $\mathbf{q}$  and departure interval  $\mathbf{t}$ , passengers choose routes according to a logit model defined on the time-expanded network.

#### 3.3.1 Capacity constraints (current status)

Although ride links are associated with nominal capacities  $u_\ell$  in the data structure, the current implementation does not incorporate capacity-dependent penalties into generalized costs. All assignment computations are therefore performed without congestion or overload effects.

Introducing capacity-aware penalties while preserving differentiability (e.g., using smooth overload functions such as softplus penalties) is a natural future extension, but it is not part of the current version of the software.

### 3.3.2 Logit routing model

Passengers are assumed to choose routes according to a logit model with dispersion parameter  $\theta > 0$ . For any path  $\mathbf{p}$ ,

$$\Pr(\mathbf{p}) \propto \exp\left(-\frac{C(\mathbf{p})}{\theta}\right), \quad C(\mathbf{p}) = \sum_{\ell \in \mathbf{p}} c_\ell.$$

When  $\theta$  is small, flows concentrate on lowest-cost paths. When  $\theta$  is large, flows spread across multiple attractive alternatives.

**Computational and behavioral motivation.** The logit formulation admits an efficient dynamic-programming implementation on the acyclic time-expanded network. It is differentiable with respect to generalized costs and, consequently, with respect to OD demand. This property is essential when the assignment model is embedded in gradient-based calibration and variational inference.

### 3.3.3 Differentiable dynamic programming formulation

Fix an OD pair  $\mathbf{q} = (\mathbf{s}_o, \mathbf{s}_d)$  and a departure interval  $\mathbf{t}$ . The *terminal* node is the destination centroid-out node  $\mathbf{s}_d^{\text{out}}$ , and the value function is initialized as

$$V(\mathbf{s}_d^{\text{out}}) = 0.$$

For any other node  $\mathbf{i}$ , let  $\mathcal{A}_\mathbf{q}^+(\mathbf{i})$  denote the set of outgoing links that are enabled for the destination group. We define

$$V(\mathbf{i}) = -\theta \log \sum_{\ell = (\mathbf{i} \rightarrow \mathbf{j}) \in \mathcal{A}_\mathbf{q}^+(\mathbf{i})} \exp\left(-\frac{c_\ell + V(\mathbf{j})}{\theta}\right).$$

This recursion is evaluated in reverse topological order using a numerically stable `logsumexp` formulation.

The probability that a passenger located at node  $\mathbf{i}$  chooses link  $\ell = (\mathbf{i} \rightarrow \mathbf{j})$  is

$$P_\ell(\mathbf{i}) = \frac{\exp\left(-\frac{c_\ell + V(\mathbf{j})}{\theta}\right)}{\sum_{(\mathbf{i} \rightarrow \mathbf{k}) \in \mathcal{A}_\mathbf{q}^+(\mathbf{i})} \exp\left(-\frac{c_{\mathbf{i}\mathbf{k}} + V(\mathbf{k})}{\theta}\right)}.$$

### 3.3.4 Flow propagation

Fix an OD pair  $\mathbf{q} = (\mathbf{s}_o, \mathbf{s}_d)$  and a departure time interval  $\mathbf{t} \in \mathcal{T}$ . The corresponding demand  $\mathbf{f}_\mathbf{q}^{\mathbf{t}}$  is injected at the origin centroid-in node  $(\mathbf{s}_o, \mathbf{t})^{\text{in}}$ .

Let  $\mathbf{y}_\mathbf{i}$  denote the flow reaching node  $\mathbf{i}$  during the forward pass. Initialization is

$$\mathbf{y}_{(\mathbf{s}_o, \mathbf{t})^{\text{in}}} = \mathbf{f}_\mathbf{q}^{\mathbf{t}}, \quad \mathbf{y}_\mathbf{i} = 0 \quad \text{for all } \mathbf{i} \neq (\mathbf{s}_o, \mathbf{t})^{\text{in}}.$$

Given transition probabilities  $P_\ell(\mathbf{i})$  on outgoing links  $\ell = (\mathbf{i} \rightarrow \mathbf{j})$ , link flows are

$$x_\ell = \mathbf{y}_\mathbf{i} P_\ell(\mathbf{i}), \quad \ell = (\mathbf{i} \rightarrow \mathbf{j}),$$

and node flows are propagated in topological order by

$$\mathbf{y}_\mathbf{j} \leftarrow \mathbf{y}_\mathbf{j} + x_\ell, \quad \ell = (\mathbf{i} \rightarrow \mathbf{j}).$$

**Destination gating.** The global graph contains egress links for all stops, but for a given OD pair  $\mathbf{q} = (\mathbf{s}_o, \mathbf{s}_d)$  the model enables only egress links leading to  $\mathbf{s}_d^{\text{out}}$  via a destination-dependent mask.



**Batched evaluation.** For computational efficiency, value functions and flow propagation are computed in batches for OD groups sharing the same destination (and, depending on the implementation, the same departure interval), enabling vectorized evaluation with JAX.

### 3.3.5 Separation of routing preparation and demand loading

For a destination group  $g$ , let  $\mathbf{d}_g$  denote its destination, let  $\mathcal{A}_g^+$  denote its enabled links after destination gating, and let  $\mathbf{f}_g$  collect the OD/time-bin demand injected for that group. The backward value recursion and the resulting transition probabilities depend on  $(\mathcal{G}, \mathbf{c}, \mathbf{d}_g, \mathcal{A}_g^+, \theta)$ , but not on  $\mathbf{f}_g$ . The assignment can therefore be written as two distinct operations:

$$\mathcal{R}_g(\mathcal{G}, \mathbf{c}, \mathbf{d}_g, \mathcal{A}_g^+, \theta) = (\mathbf{V}_g, \mathbf{P}_g),$$

followed by

$$\mathbf{x}_g = \mathcal{L}_g(\mathcal{G}, \mathbf{P}_g, \mathcal{A}_g^+, \mathbf{f}_g), \quad \mathbf{x} = \sum_g \mathbf{x}_g,$$

where  $\mathcal{R}_g$  is routing preparation and  $\mathcal{L}_g$  is the forward flow-loading operation. For fixed probabilities,  $\mathcal{L}_g$  is linear in the injected demand. This separation is exact: it does not approximate the route-choice model or alter its probabilities.

**Precomputation for fixed dispersion.** When  $\theta$  is fixed during inference, and generalized costs do not depend on the current demand, the routing state  $(\mathbf{V}_g, \mathbf{P}_g)$  is invariant across all likelihood evaluations. The software consequently prepares the effective destination masks and link probabilities once, stores them in an immutable JAX-compatible cache, and evaluates only  $\mathcal{L}_g$  as the OD parameters change. Reverse-mode differentiation then traverses the demand-loading and measurement computations, but not the invariant value-function and route-probability calculations.

This optimization is valid under the current uncongested assignment model, because link costs are fixed. It must not be applied when  $\theta$  is estimated, or if a future model makes generalized costs depend on passenger flows. In those cases, routing is recomputed inside every assignment evaluation. The ML, MAP, and variational-inference pipelines select the cached path automatically for fixed  $\theta$  and retain the dynamic path for estimated  $\theta$ .

**Cache consistency.** A prepared cache is associated with the exact graph instance, base link costs, destination-group ordering, and original destination masks from which it was constructed. These quantities are checked before cached loading is used. A change to any routing-sensitive input invalidates the cache. Empty destination sets and compact OD layouts are handled explicitly. The cached scan accumulates only total link flow and does not materialize per-destination link-flow arrays when they have not been requested.

**Computational consequences.** Let  $G$  be the number of active destination groups,  $N$  the number of graph nodes, and  $A$  the number of links. Routing preparation requires one backward pass and probability construction per group, while each subsequent cached evaluation requires only the forward loading passes. The principal incremental cache arrays scale as  $O(GA)$  for probabilities and masks. Compact OD grouping can reduce both  $G$  and this storage when frozen-zero demand removes complete destination groups.

Table 1 reports illustrative CPU measurements from the package examples. Timings are warm medians using 32-bit arithmetic and are machine-dependent; numerical equivalence is tested separately and these values are not used as test thresholds.

| Example and layout      | G  | Cache (MiB) | Forward         | Value-gradient  | Speedup |
|-------------------------|----|-------------|-----------------|-----------------|---------|
| Simple example, compact | 7  | 0.20        | 9.44 → 2.75 ms  | 12.78 → 6.21 ms | 2.06×   |
| Geneva, compact         | 18 | 3.43        | 0.449 → 0.109 s | 0.537 → 0.180 s | 2.98×   |
| Geneva, full            | 62 | 11.83       | 1.552 → 0.383 s | 1.788 → 0.591 s | 3.03×   |

Table 1: Dynamic-to-cached fixed-routing measurements. The final column reports the likelihood value-and-gradient speedup; forward speedups range from 3.43× to 4.12×.

### 3.3.6 Direct fixed-routing measurement operator

Fixed routing permits a second, more aggressive optimization when inference needs measurement predictions but not link-level outputs. For fixed transition probabilities, every forward DAG update consists only of additions and multiplication by constants. It is therefore linear in the injected OD demand. Measurement aggregation is also linear. Their composition can be represented by a matrix  $\mathbf{A}$ :

$$\boldsymbol{\lambda} = \mathbf{A}\mathbf{f}, \quad \boldsymbol{\mu} = \rho\mathbf{A}\mathbf{f}.$$

This identity is exact, rather than a local linearization.

For a compact layout containing free cells and positive frozen cells, the implemented representation separates variable columns from constants:

$$\boldsymbol{\mu} = \rho (\mathbf{A}_{\text{free}} \mathbf{f}_{\text{free}} + \mathbf{b}_{\text{fixed}}), \quad \mathbf{b}_{\text{fixed}} = \mathbf{A}_{\text{fixed}} \mathbf{f}_{\text{fixed}}.$$

Frozen-zero cells have neither columns nor an offset contribution. Operator construction proceeds destination group by destination group, but does not construct unit-demand link-flow columns. Instead it combines loading and measurement aggregation in one reverse dynamic program. For destination group  $g$ , let  $H_g(\mathbf{i}, \mathbf{m})$  be the expected accumulated contribution to measurement  $\mathbf{m}$  of one passenger starting at node  $\mathbf{i}$ . In reverse topological order,

$$H_g(\mathbf{i}, \mathbf{m}) = \sum_{\ell \in \delta^+(\mathbf{i})} P_{g\ell} [\mathbf{1}\{\ell \in \mathcal{A}_{\mathbf{m}}\} + H_g(\mathbf{h}(\ell), \mathbf{m})],$$

where  $\mathbf{h}(\ell)$  is the head of link  $\ell$ . Operator columns are obtained by gathering  $H_g$  at the OD origin nodes. Consequently, no `chunk.size`-by- $|\mathcal{A}|$  link-flow array is constructed or transferred to the host.

Only measurements mapped to links enabled for a destination group are carried by that group’s recursion. The local measurement dimension is padded to a single fixed maximum, and every OD output chunk is padded to the configured chunk size. The builder explicitly lowers and compiles this fixed-shape kernel once, then reuses the executable for every group and chunk, including the final partial chunk. For BCOO output, nonzero data and row/column indices are appended directly from returned local contributions; a dense operator is never allocated and subsequently converted.

**Construction profiling.** Each chunk reports its ordinal number, fixed shape, completed columns, elapsed time, host peak memory, kernel dispatch, device synchronization, and NumPy transfer. Final metrics record the explicit compilation count and time, number of chunks, construction and assembly time, logical dense bytes, stored bytes, nonzero count, density, host peak memory, and device peak memory when the backend exposes it. Measurement aggregation has no separately dispatched phase because it is fused into the reverse loading kernel; it is therefore not double-counted as a separate device operation.

**Validity and fallback.** The direct operator is valid only while dispersion, routing probabilities, link costs, graph structure, OD ordering, and the measurement mapping remain fixed. It is invalid when dispersion is estimated, costs depend on demand, capacity or congestion

feeds back into routing, or graph, layout, or mapping indices change. The normal fixed-routing loader remains the reference and fallback implementation and is always required when link-level outputs are requested. The estimated-dispersion path never constructs or uses the direct operator.

**Persistent cache and provenance.** Operators can be saved as compressed NumPy archives and reused by a fresh process. The cache key includes the assignment identity; content digests of the graph and all loading arrays; the complete measurement mapping; OD-layout and compact-layout fingerprints, including frozen indices and values; fixed dispersion; representation; numeric type; package version; and cache-schema version. Loaded dimensions and array shapes are also validated. Cache writes use a temporary file followed by an atomic replacement. A missing, corrupt, outdated, or provenance-mismatched artifact is rejected and rebuilt safely. The cache directory may be supplied on the estimation request or through `PUBLIC_TRANSPORTATION_OPERATOR_CACHE_DIR`. Dense and JAX BCOO representations remain available as explicit overrides.

**Activation policy.** The modes are `off`, `auto`, `dense`, and `bcoo`. Explicit modes are preserved. In automatic mode, a valid BCOO cache is reused immediately. Without a valid cache, construction is selected only when the expected number of evaluations multiplied by the measured per-evaluation saving exceeds the expected construction time. Missing construction evidence or a nonpositive saving keeps the ordinary fixed-routing loader active. Estimated dispersion always disables the operator irrespective of this policy.

**TPG structural measurement.** For the TPG case, the operator has shape  $3,690 \times 15,772$ , or 58,198,680 logical entries. Only 158,219 entries are nonzero, giving a density of 0.271860%. The logical dense float32 size is 222.01 MiB, whereas the direct BCOO representation occupies 1.811 MiB. This observed sparsity, rather than an a-priori assumption, motivates BCOO as the automatic representation.

| Chunk | Calls | Compile (s) | Build (s) | Process peak (GiB) | Warm value-gradient (s) |
|-------|-------|-------------|-----------|--------------------|-------------------------|
| 128   | 184   | 0.237       | 116.587   | 4.212              | 0.001115                |
| 256   | 124   | 0.238       | 79.264    | 4.220              | 0.001133                |
| 512   | 113   | 0.241       | 71.999    | 4.222              | 0.001075                |
| 1024  | 113   | 0.238       | 72.781    | 4.224              | 0.001164                |
| 2048  | 113   | 0.244       | 72.544    | 4.210              | 0.001104                |

Table 2: TPG direct-BCOO construction by fixed OD chunk size. The 2048 case was the largest tested safe value. At 512 and above every destination group fits in one call, so larger chunks do not reduce the 113 calls.

These measurements were rerun on the EPFL Scitas Jed server with Python 3.14.5, JAX 0.11.0, and NumPyro 0.21.0. At chunk size 1024, device synchronization accounted for 71.316 s of the 72.781 s construction; NumPy transfer took only 0.0065 s and compilation occurred exactly once. Relative to the 3.3866 s Jed reference evaluation, the measured construction break-even is approximately 21.5 evaluations. Fresh processes validated and loaded the persistent cache in 0.0203–0.0210 s. The chunk-1024 construction process peaked at 4.224 GiB RSS; the complete sequential benchmark batch peaked at 6.16 GiB.

Across three distinct demand vectors, the maximum absolute differences were 0.00390625 for the objective, 0.001953125 for the likelihood, and  $2.861 \times 10^{-6}$  for any gradient coordinate, consistent with float32 rounding. Tests also cover raw measurement predictions, frozen-zero cells, positive frozen offsets, cache round trips, corrupt-cache rebuilds, and provenance mismatch rejection.

| Iterations | Evaluations | Loader (s) | Cached BCOO (s) | Speedup | Objective difference |
|------------|-------------|------------|-----------------|---------|----------------------|
| 1          | 4           | 14.814     | 0.950           | 15.6×   | 0.0000               |
| 5          | 8           | 34.514     | 7.127           | 4.8×    | 0.0039               |
| 20         | 24          | 104.566    | 17.413          | 6.0×    | 0.0078               |

Table 3: Fresh-process TPG optimizer timings from the unchanged profiling harness on Jed. Compilation is reported separately by that harness. Warm cached BCOO value-and-gradient time was 0.001007–0.001276 s, versus 3.378727–3.657539 s for the fixed-routing loader.

### 3.3.7 Matrix-free destination streaming for very large OD layouts

The direct operator is not suitable when its dimensions or its construction state exceed the available memory. In the TPG full-network case there are 628,164 free OD cells and 426,436 measurements. A dense float32 operator would require approximately 998 GiB. More importantly, the reverse operator construction described above would require a destination-local state proportional to the number of graph nodes times the number of reachable measurements. Measurements from a sampled destination cover approximately 213,000 observations, implying about 344 GiB of value state for one destination. The explicit builder must therefore not be used for this layout.

For this regime, the software provides an exact two-pass matrix-free decomposition. Let  $\mathbf{p}_g(\mathbf{z}_g)$  be the measurement prediction contributed by destination group  $g$ , including any positive frozen-demand contribution, and let  $L$  be the nonlinear negative-binomial objective in measurement space. The complete prediction and its measurement-space cotangent are

$$\mathbf{p} = \sum_g \mathbf{p}_g(\mathbf{z}_g), \quad \mathbf{c} = \frac{\partial L(\mathbf{p})}{\partial \mathbf{p}}.$$

The gradient for each destination is then obtained independently as

$$\frac{\partial L}{\partial \mathbf{z}_g} = \mathbf{J}_{\mathbf{p}_g}(\mathbf{z}_g)^\top \mathbf{c}.$$

The implementation first streams destination predictions into the complete measurement vector. It evaluates the likelihood only after this sum is complete, then recomputes each destination under a JAX vector–Jacobian product and scatters the local result into the global free-parameter gradient. It never constructs  $\mathbf{J}_{\mathbf{p}_g}$ , a global Jacobian, or a reverse-mode tape spanning all destinations. Computing and summing separate likelihoods from partial predicted means would be mathematically incorrect and is not supported.

Frozen-zero cells are removed from each local assignment view. Positive frozen cells remain in the group and are placed at their fixed values, so their flow contribution participates in both the aggregated prediction and the common likelihood residual. Tests compare the streamed calculation with monolithic automatic differentiation for a nonlinear synthetic case with positive frozen offsets and for the actual fixed-routing loader split by destination. Values and gradients agree within explicit float32 tolerances.

The streaming implementation uses a replayable provider rather than retaining all destination predictors. It checks a declared destination-state byte ceiling before prediction, retains only compact parameter-index signatures between passes, and rejects changed group membership or ordering during replay. Thus a provider may construct and release one routing block at a time.

Six isolated full-network destinations spanning 9, 137, 276, 465, 770, and 1,181 free cells were measured under a 24 GiB process ceiling. Warm forward loading was 5.98–6.08 s and ordinary value–gradient evaluation was 12.07–12.38 s; peak resident memory was 16.02–18.16 GiB. The near-constant runtime over a 131-fold change in OD count shows that graph traversal, not demand injection, dominates. Median forward and value–gradient times were 5.991 s and

12.130 s. An ordinary sweep across 1,898 destinations is therefore estimated at 6.40 hours. Since the exact two-pass scheme adds one forward recomputation, its sequential estimate is 9.55 hours per complete objective and gradient, before optimizer overhead. Median routing preparation of 38.32 s per sample would itself require about 20.2 hours if performed serially without persistence. Consequently, a practical full-network implementation also requires direct event-aligned measurement accumulation, reduced-cost or explicit-adjoint loading, and destination-level process parallelism.

**Event-aligned measurement resolution.** Boarding and alighting mappings have additional structure that avoids generic repeated graph searches. The strict resolver formerly scanned all links for every measurement to find access links entering a matched departure event or egress links leaving a matched arrival event. It now builds node-keyed CSR indices for those relations once and retains all existing stop, time, trip/line, event-kind, uniqueness, and nonempty-link validations. On the full network, synchronized strict mapping decreased from approximately 121.2 s to 2.31 s: 0.24 s for event/link indices, 1.64 s for 426,436 record resolutions, and 0.42 s for result construction.

All 213,218 alighting measurements map to exactly one egress link. Of 213,218 boarding measurements, 207,805 map to one access link and 5,413 map to two access links because two time-bin access alternatives may enter the same departure event. The compact event-aligned representation therefore stores one primary link per measurement plus only these sparse secondary boarding pairs. It is exact; forcing every row to one link would not be. On the full-network arrays, index storage decreased from 3,454,792 to 1,749,048 bytes and warm aggregation decreased from 0.482 to 0.108 ms, with bit-identical predictions and gradients. This optimization removes mapping startup cost but does not materially change the six-second destination-loading bottleneck.

**Explicit fixed-routing adjoint.** Generic reverse-mode differentiation through the node-loading scan retains more state than is necessary when link probabilities are fixed. Let  $c_\ell$  be the cotangent of link flow and let  $a_i$  be the node-flow adjoint. A reverse topological pass evaluates

$$a_i = \sum_{\ell \in \delta^+(i)} P_\ell (c_\ell + a_{h(\ell)}).$$

The derivative with respect to initial flow injected at node  $i$  is  $a_i$ ; the existing OD injection operation then gathers this adjoint at origin nodes. The custom vector–Jacobian product therefore retains a node vector and the supplied link cotangent rather than the complete forward scan tape. It deliberately defines no derivative with respect to routing probabilities and is exposed only for the fixed-routing path.

On the representative 465-cell full-network destination, ordinary reverse mode, whole-loader rematerialization, and the explicit adjoint required respectively 12.09, 8.17, and 6.69 s for a warm value–gradient evaluation. Their resident memory immediately after that evaluation was 16.21, 17.54, and 13.91 GiB. All objective, prediction, and gradient diagnostics agreed within float32 tolerances. Whole-loader rematerialization is therefore rejected; on this CPU backend it was slower and larger than the explicit adjoint.

Combining the six-sample median forward time of 5.991 s with the 6.690 s explicit-adjoint evaluation gives a two-pass sequential estimate of 6.69 hours for 1,898 destinations, improved from 9.55 hours under ordinary autodiff but still impractical locally. The measured custom-adjoint process peak was 15.21 GiB. With 20% headroom and conservative per-worker replication, 12 workers require about 219 GiB and have an idealized lower bound of 33 minutes per complete objective and gradient. Actual server efficiency depends on per-process thread limits and memory bandwidth and must be measured on the target machine.

**Stable objective compilation and reuse.** The fixed-operator MAP objective is defined at package scope and receives all numerical operator arrays, observations, baselines, and offsets as dynamic JAX arguments. Thus parameter values, observations, and optimizer iteration limits do not alter the compiled signature; shapes, dtypes, backend, JAX/JAXLIB versions, and compilation-relevant configuration still must agree. The public `prepare_ml_objective` and `compile_ml_objective` interface separates tracing, lowering, compilation, executable loading, and first execution, and `run_ml` accepts the resulting executable without creating another jitted closure.

An isolated investigation showed that a previously observed 29.484 s first-call delay was not XLA compilation. Fixed-routing preparation had launched unused asynchronous device work before the valid operator cache was loaded, and the first objective result synchronized that work. The pipeline now validates and loads the cached operator before preparing routing. On the TPG case, isolated tracing takes about 0.026 s, lowering 0.011 s, cold XLA compilation 0.120–0.129 s, executable loading below 0.00001 s, and first execution about 0.0014 s.

The compact persistent artifact remains canonical row-major BCOO storage, with sorted unique indices. Three objective kernels were compared after all scenario preparation had completed. BCOO matvec, direct indexed scatter-add, and sorted segment reduction gave identical float32 objectives and gradients. Their warm value-and-gradient medians were respectively 0.001073 s, 0.000944 s, and 0.000973 s; direct indexed aggregation is therefore used by the stable objective.

| Iter. | Cache    | Compile/load | Warm V&G | Optimizer | Process | Objective  |
|-------|----------|--------------|----------|-----------|---------|------------|
| 1     | cold     | 0.1204       | 0.000975 | 0.303     | 5.800   | 55243.4375 |
| 1     | populate | 0.1285       | 0.000996 | 0.292     | 5.845   | 55243.4375 |
| 1     | hit      | 0.0070       | 0.000983 | 0.297     | 5.167   | 55243.4375 |
| 5     | cold     | 0.1231       | 0.001001 | 2.878     | 8.377   | 49635.4922 |
| 5     | populate | 0.1194       | 0.001021 | 2.604     | 8.167   | 49635.4922 |
| 5     | hit      | 0.0075       | 0.000958 | 2.539     | 7.622   | 49635.4922 |
| 20    | cold     | 0.1260       | 0.000991 | 5.805     | 11.623  | 49140.4258 |
| 20    | populate | 0.1260       | 0.000972 | 5.777     | 11.401  | 49140.4258 |
| 20    | hit      | 0.0070       | 0.000996 | 5.820     | 11.047  | 49140.4258 |

Table 4: Fresh-process TPG compilation-cache and optimizer measurements. The compile column is cold compilation or persistent executable loading; process time also includes scenario and assignment preparation.

The JAX persistent compilation cache is configured before computation through `PUBLIC_TRANSPORTATION_JAX_COMPILATION_CACHE_DIR` or the public package configuration function. Entry-size and compile-time thresholds default to zero so the objective is eligible. Fresh hit processes create no new cache entry and reproduce cold objectives and gradients. JAX 0.8.1 on the tested Apple CPU emits an AOT target-feature warning while loading entries even though execution succeeds and results agree exactly; this runtime warning should be rechecked when JAX is upgraded.

**Persistent assignment artifacts.** Assignment preparation is also demand-independent. Its immutable reusable state consists of the time-expanded graph arrays and labels, destination/OD grouping arrays, and generalized-cost components. It does not contain demand magnitudes, observations, priors, optimizer settings, or compiled executables. The package can store this state as compressed NPZ plus canonical JSON through optional `off`, `auto`, `refresh`, and `readonly` policies; ordinary uncached calls remain the default.

The assignment-cache provenance includes the complete graph-relevant scenario contents, OD keys, assignment and cost configuration, numeric type, package and schema versions, sentinel conventions, and indexing rules. Loaded archives are checked for exact provenance, recognized fields, counts, shapes, dtypes, CSR and padded-mask consistency, and integer/Boolean

conventions before JAX arrays are reconstructed. Cache writes use a unique temporary file, synchronization to disk, and atomic replacement. Thus concurrent writers are safe, corrupt or incompatible entries rebuild under `auto`, and `readonly` never writes. The measurement-operator cache retains its own compact-input content hash, so a valid assignment archive cannot validate an incompatible operator.

Internal TPG profiling initially assigned 3.142 s of a 3.313 s preparation to destination grouping. The cause was repeated Python scalar extraction from JAX node arrays, which introduced thousands of small device synchronizations. One NumPy view is now reused for immutable node indexing and validation, and all destination link masks are constructed in one broadcasted operation. Uncached assignment construction consequently fell to 0.097 s: graph construction took about 0.049 s, OD indexing and grouping 0.022 s, and cost construction plus synchronization 0.026 s in the compilation-cache-enabled run.

The TPG artifact contains 36,478 nodes, 11,259 links, 22,022 OD records, and 113 destination groups. Its explicit arrays occupy 6,909,418 bytes and compress to 576,227 bytes. A representative strict cache hit spent 0.00044 s opening the archive, 0.00460 s decompressing arrays, 0.01473 s in combined validation and decompression, 0.000006 s reconstructing host objects, 0.00500 s transferring and synchronizing JAX arrays, and 0.08787 s generating the canonical provenance. The complete assignment-cache call took 0.1084 s.

| Policy/case      | Assignment | Preparation | Warm V&G | Optimizer | Process |
|------------------|------------|-------------|----------|-----------|---------|
| Off              | 0.0971     | 1.1756      | 0.000992 | 0.2908    | 2.1939  |
| Populate         | 0.3412     | 1.3532      | 0.000993 | 0.2975    | 2.4106  |
| Valid hit        | 0.1084     | 1.0775      | 0.000937 | 0.2721    | 2.0211  |
| Corrupt/rebuild  | 0.3201     | 1.3339      | 0.000965 | 0.3073    | 2.4081  |
| Mismatch/rebuild | 0.3321     | 1.3463      | 0.001048 | 0.3166    | 2.3935  |
| Read-only hit    | 0.1128     | 1.1242      | 0.000972 | 0.2922    | 2.1872  |

Table 5: Fresh-process TPG assignment-cache measurements in seconds with the measurement-operator and JAX compilation caches enabled.

After eliminating scalar synchronization, uncached reconstruction is already slightly faster than strict cache validation in the assignment stage itself; canonical fingerprinting accounts for most of the hit time. The persistent artifact is therefore useful primarily for strict reproducibility and validation, while the focused grouping simplification provides the main speedup. Relative to the original profile, complete one-iteration time falls from 5.389 s to roughly 2.0–2.2 s. Cache-hit runs at 1, 5, and 20 iterations reproduce objectives 55,243.4375, 49,635.4922, and 49,140.4258 exactly.

### 3.3.8 Opt-out alternative (future extension)

In a general formulation, an opt-out alternative can be introduced for each OD pair and departure interval. In the first implementation, opt-out alternatives are not included: all demand is assumed to be assigned to the public transportation network.

## 3.4 Implementation notes for fast JAX evaluation

The time-expanded network is fixed across evaluations. All graph connectivity data are pre-computed once and stored as immutable arrays: a topological ordering of nodes; compressed representations of outgoing adjacency; and, for each link, tail/head indices and link type.

The backward recursion for the value function and forward propagation for flows are implemented using JAX primitives such as `logsumexp` and scan-like passes over the topological order. Variable out-degrees are handled through CSR-like adjacency and/or padded adjacency with masks, enabling compilation with `jit`. The implementation exposes routing preparation and demand loading as separate JAX-compatible operations. A composed operation preserves the

original dynamic assignment behavior, while the fixed-routing inference path supplies prepared probabilities directly to the loading scan.

### 3.4.1 Deterministic indexing requirement

Reproducibility of inference results requires that the indexing of nodes, links, and OD cells remain strictly deterministic across runs. Graph construction, link ordering, and OD indexing must therefore follow fixed ordering conventions (e.g., sorted iteration and deterministic array construction). Any change in indexing invalidates stored inference outputs, as the Bayesian layer relies on a fixed mapping between parameters and link indices.

## 4 Measurement Model and Likelihood

Let  $Y$  denote observed data (e.g., passenger counts on selected links, or aggregates derived from them). Operational measurements are available only for a subset of quantities. Let  $\mathcal{M}$  denote the index set of available measurements. For each  $\mathbf{m} \in \mathcal{M}$  we denote by  $Y_{\mathbf{m}}$  the observed count.

**Predicted measurement from the assignment model.** The assignment model produces link flows  $\mathbf{x}(\mathbf{f}; \theta) = \{x_{\ell}(\mathbf{f}; \theta) : \ell \in \mathcal{A}\}$ . A deterministic mapping associates each measurement  $\mathbf{m}$  with a subset of links  $\mathcal{A}_{\mathbf{m}} \subseteq \mathcal{A}$ , so that the model-predicted quantity is the sum of the corresponding link flows:

$$\lambda_{\mathbf{m}}(\mathbf{f}; \theta) = \sum_{\ell \in \mathcal{A}_{\mathbf{m}}} x_{\ell}(\mathbf{f}; \theta).$$

In the current version of the software, aggregation weights are implicitly equal to one.

**Asymmetric counting device (undercounting).** The counting device is asymmetric: it is more likely to *miss* passengers than to count passengers that are not present. We model this by introducing a detection rate  $\rho \in (0, 1]$  such that the expected observed count is

$$\mu_{\mathbf{m}}(\mathbf{f}; \theta, \rho) = \rho \lambda_{\mathbf{m}}(\mathbf{f}; \theta).$$

**Negative binomial likelihood.** Conditionally on  $(\mathbf{f}, \theta, \rho, \mathbf{r})$ , the observations are assumed independent and

$$Y_{\mathbf{m}} \mid \mathbf{f}, \theta, \rho, \mathbf{r} \sim \text{NegBin}(\mathbf{r}, p_{\mathbf{m}}(\mathbf{f}; \theta, \rho, \mathbf{r})), \quad \mathbf{m} \in \mathcal{M},$$

where  $\mathbf{r} > 0$  is a dispersion parameter and the success probability  $p_{\mathbf{m}}$  is defined so that  $\mathbb{E}[Y_{\mathbf{m}}] = \mu_{\mathbf{m}}(\mathbf{f}; \theta, \rho)$ :

$$p_{\mathbf{m}}(\mathbf{f}; \theta, \rho, \mathbf{r}) = \frac{\mathbf{r}}{\mathbf{r} + \mu_{\mathbf{m}}(\mathbf{f}; \theta, \rho)} = \frac{\mathbf{r}}{\mathbf{r} + \rho \lambda_{\mathbf{m}}(\mathbf{f}; \theta)}.$$

Hence, for  $\mathbf{y} \in \mathbb{N}_0$ ,

$$\Pr(Y_{\mathbf{m}} = \mathbf{y} \mid \mathbf{f}, \theta, \rho, \mathbf{r}) = \frac{\Gamma(\mathbf{y} + \mathbf{r})}{\Gamma(\mathbf{r}) \mathbf{y}!} (1 - p_{\mathbf{m}}(\mathbf{f}; \theta, \rho, \mathbf{r}))^{\mathbf{y}} (p_{\mathbf{m}}(\mathbf{f}; \theta, \rho, \mathbf{r}))^{\mathbf{r}}.$$

**Fixed measurement parameters.** In the current version of the software, the detection rate  $\rho$  and the dispersion parameter  $\mathbf{r}$  are treated as fixed, user-specified hyperparameters and are not estimated within the Bayesian inference procedure. Only the OD demand vector and optionally the dispersion parameter  $\theta$  are inferred. Joint estimation of  $(\rho, \mathbf{r})$  is a possible future extension.



**Likelihood and normalization.** The likelihood is the product over observed measurements:

$$\mathcal{L}(\{Y_m\}_{m \in \mathcal{M}} \mid \mathbf{f}, \boldsymbol{\theta}, \boldsymbol{\rho}, \mathbf{r}) = \prod_{m \in \mathcal{M}} \Pr(Y_m = y_m^{\text{obs}} \mid \mathbf{f}, \boldsymbol{\theta}, \boldsymbol{\rho}, \mathbf{r}).$$

The corresponding log-likelihood is

$$\ell(\mathbf{f}, \boldsymbol{\theta}, \boldsymbol{\rho}, \mathbf{r}) = \sum_{m \in \mathcal{M}} [\log \Gamma(y_m^{\text{obs}} + \mathbf{r}) - \log \Gamma(\mathbf{r}) - \log(y_m^{\text{obs}}!) + \mathbf{r} \log p_m(\mathbf{f}; \boldsymbol{\theta}, \boldsymbol{\rho}, \mathbf{r}) + y_m^{\text{obs}} \log(1 - p_m(\mathbf{f}; \boldsymbol{\theta}, \boldsymbol{\rho}, \mathbf{r}))]$$

For posterior computations, terms constant with respect to  $(\mathbf{f}, \boldsymbol{\theta})$  may be dropped.

**Alternative observation models (future extension).** Other likelihood choices (e.g., Gaussian noise on counts or Poisson models) may be implemented in the future, but the current baseline implementation uses the negative binomial likelihood described above.

## 5 Free and Frozen OD Demand Cells

An estimation run may hold a selected subset of OD/time-bin cells at predefined nonnegative values. Let  $\mathcal{I} = \{1, \dots, n_{\text{OD}}\}$  be the full assignment indexing, and partition it into the free set  $\mathcal{U}$  and the frozen set  $\mathcal{F}$ , with  $\mathcal{U} \cap \mathcal{F} = \emptyset$  and  $\mathcal{U} \cup \mathcal{F} = \mathcal{I}$ . For each  $i \in \mathcal{F}$ , let  $c_i \geq 0$  denote its predefined flow. The demand reconstruction is

$$f_i(\mathbf{z}_U) = \begin{cases} f_i^{(0)} \exp(\mathbf{z}_i), & i \in \mathcal{U}, \\ c_i, & i \in \mathcal{F}. \end{cases}$$

The smooth bounded transformation described previously is applied to the raw coordinate associated with each  $\mathbf{z}_i$ ,  $i \in \mathcal{U}$ .

The frozen cells are not represented by dummy parameters. They occur neither in the optimizer vector nor in the Bayesian guide, prior, gradient, Hessian, or covariance matrix. The full vector  $\mathbf{f}$  is reconstructed only at the boundary of the assignment model, because assignment necessarily consumes all OD cells. Thus the statistical dimension is exactly

$$d = |\mathcal{U}| + \mathbf{1}\{\boldsymbol{\theta} \text{ is estimated}\},$$

independently of  $|\mathcal{F}|$ . A frozen value may be zero or positive. In contrast, a free cell requires a strictly positive baseline  $f_i^{(0)}$ , since the multiplicative parameterization cannot move away from a zero baseline.

Frozen values are supplied by a sparse CSV file with columns `origin_stop_id`, `dest_stop_id`, `time_bin_id`, and optionally `fixed_flow`. A missing or blank `fixed_flow` denotes zero. Keys are validated against the scenario, duplicates are rejected, and the free/frozen partition is constructed in the canonical assignment order. The same immutable partition is passed to ML, MAP, and Bayesian VI. Saved results include the free indices, frozen indices, and frozen values so that downstream processing can verify the invariant. The layout also has its own canonical SHA-256 fingerprint, computed from the OD keys, partition indices, free baselines, and frozen values. This fingerprint is stored separately from the network/assignment fingerprint: changing only the frozen-demand file must therefore change the layout identity even though the network identity remains unchanged. The canonical JSON payload used to compute the digest is persisted with the result. On loading, the digest is recomputed from that payload to detect corruption or manual modification. During post-processing, the layout is rebuilt from the current scenario and `fixed_demand.csv`; its fingerprint must match the result fingerprint before assignment or report generation proceeds. The network fingerprint and layout fingerprint are checked independently.

If  $|\mathcal{U}| = 0$  and  $\theta$  is fixed, then  $\mathbf{d} = 0$ . This is treated as a deterministic evaluation rather than as an optimization or variational-inference problem. The likelihood is evaluated once for validation, while NumPyro SVI and the numerical optimizer are bypassed entirely. Returned parameter, gradient, Hessian, covariance, and posterior-coordinate arrays are correspondingly empty.

**Complexity boundary.** The software exposes a structural complexity profile so that dimensionality can be audited without relying on machine-dependent timings. For reduced dimension  $\mathbf{d}$  and effective low rank  $\mathbf{r}_q^{\text{eff}}$ , the principal statistical state sizes are

| state                        | number of scalar entries                            |
|------------------------------|-----------------------------------------------------|
| optimizer vector or gradient | $\mathbf{d}$                                        |
| diagonal Gaussian guide      | $2\mathbf{d}$                                       |
| low-rank Gaussian guide      | $2\mathbf{d} + \mathbf{d}\mathbf{r}_q^{\text{eff}}$ |
| full Gaussian guide          | $\mathbf{d} + \mathbf{d}(\mathbf{d} + 1)/2$         |
| dense Hessian or covariance  | $\mathbf{d}^2$ .                                    |

These statistical quantities depend only on the free cells and optional  $\theta$ , not on the number of frozen cells.

There is a separate deterministic cost boundary. Each likelihood evaluation passes a compact demand vector containing the  $\mathbf{n}_{\text{free}}$  free cells and the  $\mathbf{n}_{\text{fixed},+}$  strictly positive frozen cells. Frozen-zero cells are absent from that vector, from the padded OD group arrays, and from assignment demand injection. A destination group is also removed when it contains no remaining cell. Thus the assignment demand-vector size is  $\mathbf{n}_{\text{free}} + \mathbf{n}_{\text{fixed},+}$  rather than  $\mathbf{n}_{\text{OD}}$ . The time-expanded graph and other scenario-wide static arrays remain unchanged, so forward runtime still depends on the graph and on the surviving destination groups. Positive frozen flows must participate in assignment predictions; when  $\theta$  is estimated, even their route-choice probabilities change with the estimated parameter. The full  $\mathbf{n}_{\text{OD}}$  vector is reconstructed only after estimation for persistence and reporting. For every VI, ML, or MAP problem, the implementation reports the total, free, frozen-zero, and frozen-positive OD counts; the compact assignment-vector size; and the original, surviving, and removed destination-group counts. These structural metrics accompany elapsed time so that runtime differences can be interpreted in terms of the work actually performed by the assignment model.

## 6 Maximum Likelihood and Maximum A Posteriori Estimation

The differentiable assignment and measurement models can be used for point estimation without constructing a full posterior approximation. Let

$$\boldsymbol{\eta} = \begin{cases} (\tilde{\mathbf{z}}_u, \tilde{\mathbf{u}}), & \text{if } \theta \text{ is estimated,} \\ \tilde{\mathbf{z}}_u, & \text{if } \theta \text{ is fixed,} \end{cases}$$

denote the vector of raw unconstrained parameters. The same smooth transformations defined above map  $\boldsymbol{\eta}$  to the effective demand vector  $\mathbf{f}$  and, when applicable, to  $\theta$ . Consequently, all estimation methods use the same forward model and negative binomial likelihood.

### 6.1 Maximum likelihood estimation

The maximum likelihood estimator (MLE) is

$$\hat{\boldsymbol{\eta}}_{\text{ML}} = \arg \max_{\boldsymbol{\eta}} \ell(\boldsymbol{\eta}),$$

where

$$\ell(\boldsymbol{\eta}) = \log \mathcal{L}(Y \mid \mathbf{f}(\tilde{\mathbf{z}}), \boldsymbol{\theta}(\tilde{\mathbf{u}}), \boldsymbol{\rho}, \mathbf{r}),$$

with the dependence on  $\tilde{\mathbf{u}}$  omitted when  $\boldsymbol{\theta}$  is fixed. The MLE uses no prior information. The baseline demand  $\mathbf{f}^{(0)}$  still appears in the parameterization  $\mathbf{f} = \mathbf{f}^{(0)} \exp(\mathbf{z})$ , but it is not a probabilistic prior in the ML objective.

After optimization, the raw point estimate is transformed using exactly the same smooth mappings as in Bayesian inference:

$$\hat{\mathbf{z}}_q^t = z_{\max} \tanh(\hat{\mathbf{z}}_q^t / z_{\max}), \quad \hat{\mathbf{f}}_q^t = \mathbf{f}_q^{t,(0)} \exp(\hat{\mathbf{z}}_q^t),$$

and, when estimated,

$$\hat{\mathbf{u}} = \mathbf{u}_{\max} \tanh(\hat{\mathbf{u}} / \mathbf{u}_{\max}), \quad \hat{\boldsymbol{\theta}} = \exp(\hat{\mathbf{u}}).$$

## 6.2 Maximum a posteriori estimation

Maximum a posteriori (MAP) estimation uses the same likelihood and priors as the Bayesian model, but returns only the posterior mode:

$$\hat{\boldsymbol{\eta}}_{\text{MAP}} = \arg \max_{\boldsymbol{\eta}} \{\ell(\boldsymbol{\eta}) + \log \mathbf{p}(\boldsymbol{\eta})\}.$$

Here  $\mathbf{p}(\boldsymbol{\eta})$  contains the Gaussian prior on  $\tilde{\mathbf{z}}$  and, when  $\boldsymbol{\theta}$  is estimated, the Gaussian prior on  $\tilde{\mathbf{u}}$ . The priors are evaluated on the raw variables, not on their smoothly bounded transforms.

The software implements ML and MAP through the common penalized objective

$$\mathbf{Q}_w(\boldsymbol{\eta}) = -\ell(\boldsymbol{\eta}) - w \log \mathbf{p}(\boldsymbol{\eta}), \quad w \geq 0.$$

The setting  $w = 0$  gives pure maximum likelihood and  $w = 1$  gives MAP under the same prior as the Bayesian model. Intermediate or larger values provide regularized point estimates, but do not in general correspond to the posterior of the stated Bayesian model.

## 6.3 Numerical optimization and local uncertainty

The current point-estimation implementation minimizes  $\mathbf{Q}_w$  with gradient-based SciPy optimizers such as BFGS or L-BFGS-B. Gradients are supplied by JAX automatic differentiation. Because the objective need not be globally convex, convergence to the global optimum is not guaranteed; initialization, stopping tolerances, and local curvature should therefore be inspected.

When requested, the software computes the Hessian of  $\mathbf{Q}_w$  at the optimum and uses its inverse as a local covariance approximation. For ML this is an asymptotic likelihood-based approximation; for MAP it is a Laplace-style local approximation to posterior curvature. These standard errors are not equivalent to a complete posterior distribution and may be unreliable for weakly identified, asymmetric, bounded, or multimodal problems.

## 7 Bayesian Inference via Variational Inference

Let  $\tilde{\mathbf{z}}_u$  denote the vector of raw OD-deviation variables for free cells and let  $\tilde{\mathbf{u}}$  denote the raw variable associated with the dispersion parameter of the route choice model. The effective parameters supplied to the forward model are deterministic transformations of these raw variables. The posterior distribution is proportional to

$$\mathbf{p}(\tilde{\mathbf{z}}_u, \tilde{\mathbf{u}} \mid Y) \propto \mathcal{L}(Y \mid \mathbf{f}(\tilde{\mathbf{z}}_u), \boldsymbol{\theta}(\tilde{\mathbf{u}})) \mathbf{p}(\tilde{\mathbf{z}}_u) \mathbf{p}(\tilde{\mathbf{u}}).$$

When  $\boldsymbol{\theta}$  is fixed,  $\tilde{\mathbf{u}}$  and its prior are omitted.

Exact Bayesian inference is computationally intractable for high-dimensional OD vectors. The current implementation therefore uses *stochastic variational inference* (SVI) as implemented in NumPyro.

**Variational approximation.** The posterior is approximated by a parametric distribution  $q_\phi(\tilde{\mathbf{z}}_u, \tilde{\mathbf{u}})$  in the raw unconstrained parameter space. The parameters  $\phi$  are chosen to maximize the *Evidence Lower Bound* (ELBO):

$$\text{ELBO}(\phi) = \mathbb{E}_{q_\phi} [\log p(Y, \tilde{\mathbf{z}}_u, \tilde{\mathbf{u}}) - \log q_\phi(\tilde{\mathbf{z}}_u, \tilde{\mathbf{u}})].$$

Maximizing the ELBO is equivalent to minimizing  $\text{KL}(q_\phi \parallel p)$  between the variational approximation and the true posterior.

**Guide family.** The baseline implementation uses Gaussian variational families in an unconstrained parameter space (e.g., diagonal or low-rank approximations), and optimization is performed using stochastic gradients (Adam). For a requested low-rank value  $r_q$ , the implemented effective rank is

$$r_q^{\text{eff}} = \min(r_q, \mathbf{d}),$$

where  $\mathbf{d} = |\mathcal{U}| + \mathbf{1}\{\theta \text{ is estimated}\}$  is the reduced statistical dimension. This prevents a guide configured for the original model from retaining unnecessary covariance factors after many OD cells are frozen. The effective rank, rather than the oversized requested value, is recorded in the inference diagnostics. When  $\mathbf{d} = 0$ , no guide is constructed.

**Prior on  $\theta$ .** The dispersion parameter  $\theta$  is strictly positive. When it is estimated, the implementation introduces an unconstrained raw variable  $\tilde{\mathbf{u}} \in \mathbb{R}$  and defines

$$\mathbf{u} = \mathbf{u}_{\max} \tanh\left(\frac{\tilde{\mathbf{u}}}{\mathbf{u}_{\max}}\right), \quad \theta = \exp(\mathbf{u}), \quad \mathbf{u}_{\max} > 0.$$

Thus  $\theta$  is positive and smoothly bounded between  $\exp(-\mathbf{u}_{\max})$  and  $\exp(\mathbf{u}_{\max})$ . The configuration currently exposes  $\mathbf{u}_{\max}$  under the historical name `u_clip`, although no hard clipping is performed.

A Gaussian prior is placed on the raw variable,

$$\tilde{\mathbf{u}} \sim \mathcal{N}(\mu_{\tilde{\mathbf{u}}}, \sigma_{\tilde{\mathbf{u}}}^2).$$

If the requested center in effective log-dispersion space is  $\mu_{\mathbf{u}}$ —for example,  $\mu_{\mathbf{u}} = \log(\theta^{(0)})$  for a baseline value  $\theta^{(0)}$ —the raw prior center is chosen by the inverse transformation,

$$\mu_{\tilde{\mathbf{u}}} = \mathbf{u}_{\max} \operatorname{arctanh}\left(\frac{\mu_{\mathbf{u}}}{\mathbf{u}_{\max}}\right).$$

This guarantees that transforming the raw prior center gives exactly the requested effective center. It requires  $|\mu_{\mathbf{u}}| < \mathbf{u}_{\max}$ .

**Posterior transformation and reporting.** Posterior samples produced by the variational guide are samples of the raw variables  $\tilde{\mathbf{z}}$  and, when applicable,  $\tilde{\mathbf{u}}$ . Before reporting demand or  $\theta$ , the software applies the same smooth transformations used during likelihood evaluation:

$$\mathbf{z}_q^t = \mathbf{z}_{\max} \tanh(\tilde{\mathbf{z}}_q^t / \mathbf{z}_{\max}), \quad \mathbf{f}_q^t = \mathbf{f}_q^{t,(0)} \exp(\mathbf{z}_q^t),$$

and

$$\mathbf{u} = \mathbf{u}_{\max} \tanh(\tilde{\mathbf{u}} / \mathbf{u}_{\max}), \quad \theta = \exp(\mathbf{u}).$$

Consequently, reported posterior summaries correspond exactly to the parameter values used by the assignment and likelihood, rather than to untransformed raw samples.

**Alternative inference engines.** Markov chain Monte Carlo methods could be used in principle, but they are computationally expensive in high-dimensional OD settings. Support for alternative probabilistic programming frameworks (e.g., PyMC) may be considered in the future; the present baseline is NumPyro+SVI.

## 8 Comparison of the Estimation Methods

The three estimation approaches share the assignment model, measurement equation, likelihood, raw parameter space, and smooth parameter transformations. They differ in the statistical quantity they target.

**ML versus MAP.** The MLE maximizes the likelihood alone, whereas MAP balances the likelihood with the specified prior. They therefore need not coincide. MAP is typically pulled toward the prior center and can be more stable in weakly identified problems. As the amount of informative data increases and the likelihood dominates the prior, the ML and MAP estimates will often become close, subject to regularity and identifiability conditions.

**MAP versus Bayesian inference.** If MAP and Bayesian inference use exactly the same likelihood, priors, and parameterization, the MAP estimate is the mode of the exact Bayesian posterior. It is not generally equal to the posterior mean or median. Equality between the mode and mean occurs only in special cases, such as an exactly symmetric unimodal posterior. The difference may be substantial for skewed, weakly identified, bounded, or multimodal posteriors.

**Variational approximation.** The implemented Bayesian procedure reports means and other summaries computed from the variational approximation  $\mathbf{q}_\phi$ , not from the exact posterior. Differences between MAP and VI results can therefore arise from three distinct sources:

1. the intrinsic difference between a posterior mode and a posterior mean;
2. approximation error introduced by the selected variational guide; and
3. optimization and finite posterior-sampling error in the VI procedure.

The posterior draws used to compute VI summaries introduce Monte Carlo variability, but this is not the only, and often not the dominant, reason for differences between the estimates.

**When estimates should be close.** Under standard regularity conditions, with a large informative sample and a well-identified model, the posterior is often approximately Gaussian and concentrated. In that regime the MLE, MAP estimate, posterior mode, and posterior mean may be close. This is an asymptotic property rather than an equality guaranteed by the software.

**Recommended comparisons.** For validating the implementation, pure ML should be compared across optimizers using  $\mathbf{w} = 0$ , and MAP should be compared with the mode of the same posterior using  $\mathbf{w} = 1$ . Comparing an ML or MAP point estimate directly with the reported VI posterior mean is useful diagnostically, but disagreement should not automatically be interpreted as a software error. In every comparison, the effective transformed values  $(\mathbf{f}, \boldsymbol{\theta})$  should be compared rather than the raw variables  $(\tilde{\mathbf{z}}, \tilde{\mathbf{u}})$ .

## 9 Discussion

This software framework combines timetable information and operational counts to infer OD demand and quantify uncertainty, while using a differentiable assignment model compatible with ML, MAP, and gradient-based Bayesian inference. ML and MAP provide computationally convenient point estimates and local curvature diagnostics, whereas Bayesian VI additionally provides an approximate distribution that represents parameter uncertainty.

Planned extensions of the software include incorporating capacity effects in a differentiable manner, estimating additional measurement parameters (e.g.,  $\rho$  and  $\mathbf{r}$ ), supporting weighted measurement aggregations, and adding opt-out alternatives.

## References

Dial, R. B. (1971). A probabilistic multipath traffic assignment model which obviates path enumeration, *Transportation research* **5**(2): 83–111.